

Birla Institute of Technology and Science, Pilani

Work Integrated Learning Programmes Division

M. Tech. in AI & ML

I Semester 2025-2026

Mid-Semester Test

(EC2 - Makeup)

Course Number	AIMLCZG511
Course Name	DEEP NEURAL NETWORK
Nature of Exam	Closed Book
Weight-age for grading	30
Total Marks	100
Duration	2 hrs

# Pages	27
# Questions	5

-
1. (a) For an XOR (exclusive OR) logical function, consider plotting the four points on a 2D coordinate plane where the x-axis represents x_1 and the y-axis represents x_2 . The class 0 points (output = 0) are located at coordinates (0,0) and (1,1). The class 1 points (output = 1) are located at coordinates (0,1) and (1,0).
- (i) Attempt to draw a single straight line on this 2D plane that separates all class 0 points on one side from all class 1 points on the other side. Can you successfully draw such a line? Explain clearly why or why not, referring to the spatial arrangement of the four points. [3 marks]
- (ii) What fundamental limitation of the perceptron model does this reveal about its ability to learn certain types of functions? [3 marks]
- (b) A student trains perceptrons to learn three different logical functions using identical initialization ($\mathbf{w} = [0, 0, 0]^T$) and learning rate ($\eta = 0.1$). The training results are:
- AND gate:** Converges to 100% accuracy in 6 epochs.
- OR gate:** Converges to 100% accuracy in 4 epochs.
- XOR gate:** After 100 epochs, remains stuck at 50% accuracy (random guessing).
- Evaluate why these three functions exhibit such different learning behaviors. Your answer should explain:
- (i) Why XOR cannot reach 100% accuracy while AND and OR can? [3 marks]
- (ii) Why OR converges faster than AND (4 epochs versus 6 epochs), considering that OR has three positive examples while AND has only one. [3 marks]
- (c) The following Python code is intended to train and evaluate a perceptron. However, it contains 3 logical errors.
- For each error: (i) Identify the line number and describe what is wrong, (ii) Write the corrected code. After finding all errors, (iii) explain what would happen during training if these errors remain uncorrected. [8 marks]

```

1 import numpy as np
2
3 def perceptron_train(X, y, lr=0.1, epochs=10):
4     """X: (N x d) with bias, y: (N,) labels {0,1}"""
5     N, d = X.shape
6     weights = np.zeros(d)
7
8     for epoch in range(epochs):
9         for i in range(N):
10             z = np.dot(X[i], weights)
11             y_pred = 1 if z > 0 else 0
12
13             if y_pred != y[i]:
14                 error = y[i] - y_pred
15                 weights = weights - lr * error * X[i]
16     return weights
17
18 # Test on AND gate
19 X = np.array([[1, 0, 0], [1, 0, 1], [1, 1, 0], [1, 1, 1]])
20 y = np.array([0, 0, 0, 1])
21 w = perceptron_train(X, y)
22 preds = (np.dot(X, w) > 0).astype(int)
23 acc = np.mean(preds == y)

```

2. (a) A linear regression model predicts student exam scores based on two features: hours spent studying and hours of sleep. Using a dataset of three students, the gradient has been computed as $\nabla J = \begin{bmatrix} 5 & 30 & 35 \end{bmatrix}^\top$. The current weights are $\mathbf{w} = \begin{bmatrix} 20 & 8 & 5 \end{bmatrix}^\top$.
- (i) Perform one weight update using learning rate $\eta = 0.01$. Show all three updated weight values. [2 marks]
- (ii) Now perform the update again starting from the same initial weights $\mathbf{w} = \begin{bmatrix} 20 & 8 & 5 \end{bmatrix}^\top$, but this time using learning rate $\eta = 1.0$. Show all three updated weight values. [2 marks]
- (iii) Compare the magnitude of change in both cases. Which learning rate causes a larger change? What potential problems could arise from using too large a learning rate? Explain with reference to convergence behavior. [2 marks]
- (b) A real estate company wants to deploy a linear regression model to provide instant online property valuations. They have two models:
- Model A:** Uses 2 features (area, location score). Training RMSE is 35,000. Model is simple, easily explainable to customers. Predictions can be justified: "Your home is valued at X because of Y sq ft and location score Z."
- Model B:** Uses 15 features (area, location, age, bedrooms, bathrooms, garage, school rating, crime rate, etc.). Training RMSE is 28,000. Model is more accurate but predictions are harder to explain to customers who question valuations.
- Evaluate which model the company should deploy on their public website. In your evaluation, consider:
- (i) The 7,000 RMSE improvement represents a 20% reduction in prediction error - is this improvement significant enough for real estate transactions? [2 marks]
- (ii) The importance of customer trust and transparency in real estate, where buyers and sellers often question automated valuations and need clear justifications. [2 marks]
- (iii) Legal and regulatory requirements in some regions that mandate explainability of automated decision systems, particularly for high-value transactions like real estate. [2 marks]

- (c) The following code implements gradient descent for linear regression. It contains 3 logical errors.

For each error: (i) Identify and describe the error, (ii) Provide the correction. After corrections, (iii) explain what symptoms would appear during training with these errors and why the model would fail to converge properly. [8 marks]

```
1 import numpy as np
2
3 def predict(X, weights):
4     return np.dot(X, weights)
5
6 def mse_loss(y_true, y_pred):
7     residuals = y_pred - y_true
8     return np.mean(residuals)
9
10 def gradient_descent_step(X, y, weights, lr):
11     y_pred = predict(X, weights)
12     error = y_pred - y
13     gradient = np.dot(X, error) / len(y)
14     return weights + lr * gradient
15
16 def compute_residuals(X, y, weights):
17     y_pred = predict(X, weights)
18     return y - y_pred
```

3. (a) A binary logistic regression classifier operates on 2D input data. The model has learned weights $\mathbf{w} = [-1 \ 0.5 \ 0.8]^\top$.

(i) Consider two test points: $(x_1 = 0, x_2 = 0)$ and $(x_1 = 4, x_2 = 2)$. For each point, calculate the weighted sum z and determine whether $\hat{y} = \sigma(z) \geq 0.5$. Which point is classified as class 1? [2 marks]

(ii) Consider a customer purchase prediction model with weights $\mathbf{w} = [-2 \ 0.05 \ 0.03]^\top$. For a customer with age=30 and income=50, the predicted probability is $\hat{y} \approx 0.62$. Using classification threshold = 0.5, what class is predicted? IF we use threshold = 0.7, what class is predicted? [2 marks]

(iii) If the true class for this customer is 1 (purchase), which threshold produces the correct prediction? Explain the trade-off: what happens to false positives and false negatives when you increase the classification threshold from 0.5 to higher values? [2 marks]

- (b) A healthcare startup develops a logistic regression model to identify patients at risk for diabetes using routine checkup data (BMI, age, blood pressure, glucose). Two models are compared:

Model A (Conservative): Uses threshold = 0.3 (flags patient as "at-risk" if $\hat{y} \geq 0.3$).

- Identifies 85% of true diabetic/pre-diabetic patients (high recall)
- Flags 40% of healthy patients as false positives
- Results in many unnecessary follow-up tests (200 each in local currency)

Model B (Balanced): Uses threshold = 0.5 (standard threshold).

- Identifies 65% of true diabetic/pre-diabetic patients (moderate recall)
- Flags 15% of healthy patients as false positives
- Misses 35% of at-risk patients who don't get early intervention

Evaluate which model the startup should deploy in their preventive health app. In your evaluation, consider:

(i) The medical and financial cost of missing early diabetes detection, including long-term complications (kidney disease, cardiovascular issues) and expensive late-stage treatment. [2 marks]

(ii) The burden on the healthcare system and patient anxiety from excessive false positives, including unnecessary testing costs and psychological impact. [2 marks]

(iii) The ethical responsibility in preventive healthcare - is it better to over-diagnose or under-diagnose a serious but manageable chronic condition? [2 marks]

- (c) The following code implements logistic regression with BCE loss. It contains 3 logical errors.

For each error: (i) Identify the line number and describe the error, (ii) Write the corrected code. Finally, (iii) explain what would happen to the training process if Error 2 (incomplete BCE loss) remains uncorrected. How would this affect the learned decision boundary? [8 marks]

```
1 import numpy as np
2
3 def sigmoid(z):
4     return 1 / (1 - np.exp(-z))
5
6 def bce_loss(y_true, y_pred):
7     eps = 1e-15
8     y_pred = np.clip(y_pred, eps, 1-eps)
9     return -np.mean(y_true * np.log(y_pred))
10
11 def train_step(X, y, weights, lr):
12     z = np.dot(X, weights)
13     y_pred = sigmoid(z)
14     gradient = np.dot(X.T, (y - y_pred)) / len(y)
15     return weights - lr * gradient
16
17 def predict_class(X, weights, threshold=0.5):
18     probs = sigmoid(np.dot(X, weights))
19     return (probs >= threshold).astype(int)
```

4. (a) A 3-class softmax classifier processes an input and produces logit values $\mathbf{z} = [2.0 \ 0.5 \ 1.5]^\top$ for classes 0, 1, and 2 respectively. The true label is class 2, represented in one-hot encoding as $\mathbf{y} = [0 \ 0 \ 1]^\top$.
- (i) Compute the exponential e^{z_k} for each class, calculate their sum $\sum_{j=0}^2 e^{z_j}$, then compute softmax probabilities $\hat{y}_k = \frac{e^{z_k}}{\sum_{j=0}^2 e^{z_j}}$ for all three classes. Which class does the model predict? [4 marks]
- (ii) Calculate the Categorical Cross-Entropy (CCE) loss using $L = -\sum_{k=0}^2 y_k \log(\hat{y}_k)$. The model predicts class 0 but the true label is class 2. What does the magnitude of the loss value reveal about the model's confidence in its incorrect prediction? Explain the relationship between prediction confidence and loss. [4 marks]
- (b) A product review classifier uses softmax regression to categorize reviews into 4 sentiment classes: Very Negative (0), Negative (1), Positive (2), Very Positive (3). The confusion matrix on 800 test reviews is:

	Pred 0	Pred 1	Pred 2	Pred 3	Total
True 0	85	12	2	1	100
True 1	18	220	50	12	300
True 2	3	45	180	22	250
True 3	0	8	30	112	150
Total	106	285	262	147	800

Analyze: (i) Calculate overall accuracy. Then calculate the precision for class 1 (Negative). Show your calculation with TP and FP clearly identified from the confusion matrix. [4 marks]

(ii) Observe that many reviews in class 1 (Negative) are misclassified as class 2 (Positive) - 50 cases. Similarly, many class 2 reviews are misclassified as class 1 - 45 cases. What does this mutual confusion between opposite sentiments suggest about the model's learned decision boundaries? Is this a serious problem for a business using this classifier to analyze customer feedback? [2 marks]

- (c) The following code implements softmax classification. It contains 3 logical errors. For each error: (i) Identify the line number and error, (ii) Write correction. Finally, (iii) explain what would happen during prediction if Error 3 remains uncorrected . [6 marks]

```
1 import numpy as np
2
3 def softmax(Z):
4     exp_Z = np.exp(Z)
5     return exp_Z / np.sum(exp_Z, axis=0, keepdims=True)
6
7 def cce_loss(Y_true, Y_pred):
8     eps = 1e-15
9     Y_pred = np.clip(Y_pred, eps, 1-eps)
10    return np.mean(Y_true * np.log(Y_pred))
11
12 def predict_classes(X, W):
13     Z = np.dot(X, W)
14     probs = softmax(Z)
15     return np.argmax(probs, axis=1)
16
17 def compute_accuracy(y_true, y_pred):
18     return np.mean(y_true == y_pred)
```


5. (a) Consider a hidden layer in a neural network that receives pre-activation values $\mathbf{z} = [-2.0 \ 0.0 \ 1.5]^\top$ for three neurons.

(i) Apply the ReLU activation function $\text{ReLU}(z) = \max(0, z)$ to each component of $\mathbf{z}$. Show the output value for each of the three neurons after activation. [2 marks]

(ii) Now apply the Sigmoid activation function $\sigma(z) = \frac{1}{1+e^{-z}}$ to each component of $\mathbf{z}$. Calculate the output for each neuron (you may round to 3 decimal places). Based on your results, which activation function (ReLU or Sigmoid) preserves information about negative input values? Explain briefly how this affects the network's ability to learn. [3 marks]

- (b) Design a DFNN with one hidden layer for handwritten digit classification (MNIST-style) with specifications:

- Input: $28 \times 28 = 784$ pixel values (flattened grayscale images)
- Output: 10 classes (digits 0-9)
- Training data: 10,000 samples
- Constraint: Total parameters must not exceed 20,000

Analyze: (i) Write the formula for total parameters including biases in terms of n_h , the hidden layer size. Set up inequality $\leq 20,000$ and solve for maximum n_h . [2.5 marks]

(ii) Choose a reasonable n_h value less than your maximum. Calculate exact parameters. Is this reasonable given 10,000 training samples? Explain the risk if n_h is too large relative to training data size. [2.5 marks]

- (c) A smartphone voice assistant uses a DFNN for real-time voice activity detection (speech vs silence/noise), running entirely on-device for privacy. Two architectures:

Architecture A: $[40 \rightarrow 16 \rightarrow 8 \rightarrow 1]$, 1,000 params, 0.8ms inference, 94% accuracy, 4KB memory.

Architecture B: $[40 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 1]$, 4,500 params, 3.2ms inference, 96% accuracy, 18KB memory.

The system must process 100 audio segments per second (10ms budget per segment) and run continuously.

Evaluate which to deploy considering:

(i) Real-time constraint (10ms budget) and continuous operation impact on battery life. [2 marks]

(ii) Whether 2% accuracy improvement justifies $4\times$ slower inference and higher power consumption. [2 marks]

- (d) The following code implements backpropagation for a 2-layer network. It contains 3 logical errors. For each error: (i) Identify the line number and mistake, (ii) Correct it. Finally, (iii) explain what happens to gradient flow if Error 2 (wrong sign in dZ2) remains uncorrected . [6 marks]

```
1 import numpy as np
2
3 def relu(Z):
4     return np.maximum(0, Z)
5
6 def relu_derivative(Z):
7     return (Z >= 0).astype(float)
8
9 def backward_pass(X, Y, cache, W2):
10    Z1, A1, Z2, A2 = cache
11    N = X.shape[0]
12
13    dZ2 = A2 + Y
14    dW2 = np.dot(A1.T, dZ2) / N
15
16    dA1 = np.dot(dZ2, W2.T)
17    dZ1 = dA1 * relu_derivative(Z1)
18    dW1 = np.dot(X.T, dZ1) / N
19
20    return dW1, dW2
21
22 def update_weights(W1, W2, dW1, dW2, lr):
23    W1 = W1 + lr * dW1
24    W2 = W2 - lr * dW2
25    return W1, W2
```

1. (a) **Solution:**

- i. No, you cannot draw a single straight line that separates all class 0 points from class 1 points.

Explanation: When plotting the four XOR points:

- Class 0: (0,0) and (1,1) - located at opposite corners of a unit square
- Class 1: (0,1) and (1,0) - located at the other two opposite corners

Any straight line that separates (0,0) from (0,1) and (1,0) will also separate (1,1) on the wrong side. The points are arranged diagonally such that no linear boundary can separate them correctly.

Marking Rubric:

- Correct answer (No): 0.5 marks
 - Identification of point positions: 1 mark
 - Clear explanation of why linear separation fails: 1.5 marks
- ii. This demonstrates the fundamental limitation that **perceptrons can only learn linearly separable functions**. A perceptron implements a linear decision boundary (hyperplane), and XOR requires a non-linear decision boundary to separate the classes correctly.

The perceptron cannot learn XOR because:

- It can only represent functions of the form $\hat{y} = \text{step}(w_0 + w_1x_1 + w_2x_2)$
- This defines a straight line decision boundary in 2D space
- XOR requires at least two linear boundaries or a curved boundary
- This limitation necessitates multi-layer networks to solve non-linearly separable problems

Marking Rubric:

- Statement of linear separability limitation: 1 mark
- Explanation of why perceptron is limited to linear boundaries: 1 mark
- Connection to need for multi-layer networks: 1 mark

(b) **Solution:**

- i. XOR is **not linearly separable**, while both AND and OR are linearly separable.

AND gate analysis:

- Only (1,1) $\rightarrow$ 1, all others $\rightarrow$ 0. Points (0,0), (0,1), (1,0) on one side, (1,1) on the other

OR gate analysis:

- (0,1), (1,0), (1,1) $\rightarrow$ 1, only (0,0) $\rightarrow$ 0. Point (0,0) on one side, all others on the other

XOR gate analysis:

- (0,1), (1,0) $\rightarrow$ 1 but (0,0), (1,1) $\rightarrow$ 0
- No single line can separate diagonal patterns
- Perceptron convergence theorem guarantees convergence only for linearly separable data
- XOR violates this requirement, so perceptron cannot converge

Marking Rubric:

- Correct identification of linear separability: 1 mark
- Explanation for AND/OR being learnable: 1 mark

- Explanation for XOR being unlearnable with reference to perceptron limitations: 1 mark
- ii. OR converges faster (4 epochs vs 6 epochs) because:
- Reason 1: More positive examples With more positive examples, OR receives more frequent weight updates pushing the decision boundary in the correct direction
- Reason 2: Easier decision boundary
- Reason 3: Weight update frequency

Marking Rubric:

- Identification of difference in positive example count: 1 mark
- Explanation of how this affects weight updates: 1 mark
- Additional insight about decision boundary placement or update frequency: 1 mark

(c) **Error 1 (Line 11): Step activation threshold incorrect**

Identification [1 mark]: The condition $z > 0$ should be $z \geq 0$.

Correction [1.5 marks]:

```
y_pred = 1 if z >= 0 else 0
```

Error 2 (Line 15): Weight update sign reversed

Identification [1 mark]: The perceptron learning rule should ADD the correction term, not subtract it.

Correction [1.5 marks]:

```
weights = weights + learning_rate * error * X[i]
```

Error 3 (Line 22): Prediction threshold inconsistent [2.5 marks]

Identification [1 mark]: The prediction uses > 0 but training uses > 0 (after we fix Error 1 to ≥ 0). For consistency, predictions should also use ≥ 0 .

Correction [1.5 marks]:

```
preds = (np.dot(X, w) >= 0).astype(int)
```

(iii) Impact of uncorrected errors

If these errors remain:

- **Error 1 + Error 3:** Points exactly on the decision boundary ($z = 0$) would be classified as 0 during training but we'd be inconsistent during prediction. This causes training-test mismatch.
- **Error 2 (most critical):** The reversed sign means weights move in the OPPOSITE direction of the correct gradient. Instead of correcting mistakes, the perceptron reinforces them. For a misclassified example where $y = 1$ but $\hat{y} = 0$, the error is +1, but subtracting pushes weights AWAY from the correct classification. The model will diverge or oscillate rather than converge.

Marking Rubric:

- Identification of threshold inconsistency issue: 0.5 marks
- Explanation of reversed gradient problem: 1.5 marks
- Clear description of failure to converge: 0.5 marks

2. (a) **Solution:**

- i. Update with $\eta = 0.01$ [2 marks]

$$\begin{aligned}\mathbf{w}^{new} &= \mathbf{w} - \eta \nabla J \\ &= \begin{bmatrix} 20 \\ 8 \\ 5 \end{bmatrix} - 0.01 \begin{bmatrix} 5 \\ 30 \\ 35 \end{bmatrix} = \begin{bmatrix} 19.95 \\ 7.70 \\ 4.65 \end{bmatrix}\end{aligned}$$

Marking Rubric:

- Correct formula application: 0.5 marks
 - All three weight calculations correct: 1.5 marks (0.5 each)
- ii. Update with $\eta = 1.0$ [2 marks]

$$\begin{aligned}\mathbf{w}^{new} &= \mathbf{w} - \eta \nabla J \\ &= \begin{bmatrix} 20 \\ 8 \\ 5 \end{bmatrix} - 1.0 \begin{bmatrix} 5 \\ 30 \\ 35 \end{bmatrix} = \begin{bmatrix} 15 \\ -22 \\ -30 \end{bmatrix}\end{aligned}$$

Marking Rubric:

- Correct formula application: 0.5 marks
 - All three weight calculations correct: 1.5 marks (0.5 each)
- iii. Comparison and analysis [2 marks]

Magnitude comparison:

- $\eta = 0.01$: Changes are $[-0.05, -0.30, -0.35]$ (small adjustments)
- $\eta = 1.0$: Changes are $[-5, -22, -30]$ (large jumps)
- The learning rate $\eta = 1.0$ causes $100\times$ larger changes

Problems with large learning rates:

- Overshooting: Large steps can jump over the minimum, causing oscillation around the optimal solution without converging
- Divergence: If the learning rate is too large relative to the loss curvature, weights can move away from the minimum, causing loss to increase rather than decrease

Marking Rubric:

- Correct identification that $\eta = 1.0$ causes larger changes: 0.5 marks
- Clear explanation of overshooting/divergence: 1.5 mark

(b) **Recommendation: Model A should be deployed**

- i. Significance of \$7,000 RMSE improvement [2 marks]
The 20% error reduction is notable but may not be decisive:

Context of error magnitude:

- For a typical home valued at \$350,000:
 - Model A error: $\pm 35,000$ (10% of value)
 - Model B error: $\pm 28,000$ (8% of value)
- The 7,000 difference is about 2% of home value
- Both models still have substantial uncertainty

Practical impact:

- Real estate decisions rarely hinge on 7,000 differences
- Both models provide reasonable ballpark estimates
- The improvement doesn't justify sacrificing explainability

Marking Rubric:

- Contextualizing 7,000 relative to home values: 1 mark
- Discussing whether improvement is decisive: 1 mark

ii. Customer trust and transparency [2 marks]

Real estate is a high-stakes, trust-dependent transaction:

Transparency value:

- Ability to explain "Your home is worth X because of 1,800 sq ft and excellent location" builds confidence
- Customers can challenge or verify these factors independently
- Black-box models create skepticism: "Why does your system say 350K?"

Business implications:

- Dissatisfied customers may seek second opinions from competitors
- Unexplainable valuations increase customer service burden
- Simple models help agents justify valuations to skeptical clients
- Trust is more valuable than marginal accuracy in real estate

Marking Rubric:

- Discussion of transparency and customer trust: 1 mark
- Business/practical implications: 1 mark

iii. Legal and regulatory requirements [2 marks]

Automated valuation systems face increasing scrutiny:

Regulatory landscape:

- EU's GDPR includes "right to explanation" for automated decisions
- US fair lending laws require demonstrable absence of bias
- Some jurisdictions mandate explainability for property valuations
- Financial services increasingly regulated for algorithmic transparency

Legal risks with Model B:

- Cannot explain how 15 features combine to produce valuation
- Difficult to audit for discriminatory patterns (e.g., crime rate proxy for race)
- May not satisfy regulatory requirements in some markets
- Legal liability if model cannot be explained in disputes

Compliance advantage of Model A:

- Simple model is inherently auditable
- Easy to verify no protected characteristics are used
- Can demonstrate model logic to regulators
- Reduces legal risk in contested valuations

Marking Rubric:

- Mention of legal/regulatory requirements for explainability: 1 mark
- Discussion of compliance advantages or risks: 1 mark

(c) **Error 1 (Line 8): MSE formula missing squaring**

Identification [1 mark]: The MSE formula requires squaring the residuals.

Correction [1.5 marks]:

```
return np.mean(residuals**2)
OR
return np.mean((y_pred - y_true)**2)
```

Error 2 (Line 13): Missing transpose on X

Identification [1 mark]: The transpose is missing.

Correction [1.5 marks]:

```
gradient = np.dot(X.T, error) / len(y)
```

Error 3 (Line 14): Wrong sign in weight update

Identification [1 mark]: Gradient descent requires moving in the direction opposite to the gradient (downhill).

Correction [1.5 marks]:

```
return weights - lr * gradient
```

(iii) Training symptoms with these errors

- Error 1 impact:
 - Loss doesn't reflect actual prediction quality
 - Cannot track convergence properly
 - Monitoring becomes meaningless
- Error 2 impact:
 - Dimension mismatch: $(N \times d) \times (N \times 1)$ is undefined for $N \neq d$
 - Code will crash with "shapes not aligned" error
 - Training cannot proceed at all
 - This is the most critical error - prevents any training
- Error 3 impact:
 - Weights move uphill (increasing loss) instead of downhill
 - Loss diverges to infinity rather than converging
 - Model performance worsens with each iteration
 - Training is completely counterproductive

Marking Rubric:

- Identification of dimension mismatch from Error 2: 0.75 marks
- Explanation of divergence from Error 3: 1 mark
- Discussion of loss function incorrectness: 0.75 marks

3. (a) i. Test points classification

For point $(x_1 = 0, x_2 = 0)$:

$$z = w_0 + w_1x_1 + w_2x_2 = -1 + 0.5(0) + 0.8(0) = -1$$

$$\hat{y} = \sigma(-1) = \frac{1}{1 + e^1} \approx 0.27 < 0.5$$

Therefore, $(0, 0)$ is classified as **class 0**.

For point $(x_1 = 4, x_2 = 2)$:

$$z = w_0 + w_1x_1 + w_2x_2 = -1 + 0.5(4) + 0.8(2) = 2.6$$

$$\hat{y} = \sigma(2.6) = \frac{1}{1 + e^{-2.6}} \approx 0.93 > 0.5$$

Therefore, $(4, 2)$ is classified as **class 1**.

Marking Rubric:

- Correct calculation of z for both points: 1 mark (0.5 each)
- Correct classification determination: 1 mark (0.5 each)

- ii. Given: $\hat{y} = 0.62$ for the customer.

With threshold = 0.5: Since $\hat{y} = 0.62 > 0.5$, predict **class 1 (purchase)**.

With threshold = 0.7: Since $\hat{y} = 0.62 < 0.7$, predict **class 0 (no purchase)**.

Marking Rubric:

- Correct prediction with threshold 0.5: 0.5 marks
- Correct prediction with threshold 0.7: 0.5 marks
- Clear reasoning shown: 1 mark

- iii. Since the true class is 1 (purchase) and $\hat{y} = 0.62$:

- Threshold 0.5 produces **correct prediction** (predicts 1, truth is 1)
- Threshold 0.7 produces **incorrect prediction** (predicts 0, truth is 1)

Trade-offs when increasing threshold from 0.5 to higher values:

Higher threshold = Higher precision but lower recall. Use higher thresholds when false positives are more costly than false negatives.

Marking Rubric:

- Correct identification of which threshold works: 0.5 marks
- Clear explanation of FP decreasing and FN increasing: 1 mark
- Additional insight on precision/recall trade-off: 0.5 marks

(b) **Recommendation: Model A (Conservative) should be deployed**

i. Cost of missing early diabetes detection

- Medical consequences: Disease progression, Long-term complications like Kidney disease , Mortality risk
- Financial consequences: Treatment costs escalate dramatically. Model B misses 35% of at-risk patients: If serving 10,000 patients with 500 at-risk, Model B misses 175 patients. At 30,000 average additional cost over 5 years, that's 5.25 million in preventable costs.

Marking Rubric:

- Discussion of medical complications from missed detection: 1 mark
- Quantification or discussion of financial costs: 1 mark

ii. Burden of excessive false positives

- Healthcare system burden
 - Unnecessary testing: Model A flags 40% of healthy patients (false positives)
- Patient psychological impact
- Cost-benefit comparison
 - Model A: 760,000 in excess testing vs preventing 5.25 million in complications
 - Model B: Saves 760,000 in testing but allows 5.25 million in preventable complications
 - Net benefit of Model A: approximately 4.5 million saved

Marking Rubric:

- Discussion of healthcare system burden or testing costs: 1 mark
- Discussion of patient anxiety and psychological impact: 1 mark

iii. Ethical responsibility in preventive healthcare

- **Over-diagnosis (Model A):** Causes temporary inconvenience and moderate cost
- **Under-diagnosis (Model B):** Causes permanent health damage and high cost
- For serious but treatable conditions, over-diagnosis is ethically preferable
- App should clearly communicate it's a screening tool, not diagnostic
- Transparency about 40% false positive rate maintains trust

Marking Rubric:

- Discussion of preventive healthcare philosophy: 1 mark
- Analysis of asymmetric consequences (over-diagnosis vs under-diagnosis): 1 mark

(c) **Error 1 (Line 4): Sigmoid denominator has wrong sign**

Identification [1 mark]: The sigmoid function formula

Correction [1.5 marks]:

```
return 1 / (1 + np.exp(-z))
```

Error 2 (Line 9): BCE loss incomplete

Identification [1 mark]:

Correction [1.5 marks]:

```
return -np.mean(y_true * np.log(y_pred) +  
                (1 - y_true) * np.log(1 - y_pred))
```

Error 3 (Line 14): Gradient sign reversed [2.5 marks]

Identification [1 mark]:

Correction [1.5 marks]:

```
gradient = np.dot(X.T, (y_pred - y)) / len(y)
```

(iii) Impact if Error 2 remains uncorrected

If the incomplete BCE loss remains:

- Only penalizes errors on positive class ($y = 1$)
- No penalty for misclassifying negative class ($y = 0$)
- Model learns to optimize only for correctly classifying class 1
- Decision boundary shifts to maximize recall for class 1
- Model becomes biased toward predicting class 1

Marking Rubric:

- Identification that negative class is not penalized: 1 mark
- Explanation of decision boundary shift: 1 mark
- Discussion of practical consequences (bias toward class 1): 0.5 marks

4. (a) i. Softmax probabilities and prediction

Step 1: Compute exponentials

$$e^{z_0} = e^{2.0} \approx 7.389$$

$$e^{z_1} = e^{0.5} \approx 1.649$$

$$e^{z_2} = e^{1.5} \approx 4.482$$

Step 2: Calculate sum

$$\sum_{j=0}^2 e^{z_j} = 7.389 + 1.649 + 4.482 = 13.520$$

Step 3: Compute softmax probabilities

$$\hat{y}_0 = \frac{e^{z_0}}{\sum e^{z_j}} = \frac{7.389}{13.520} \approx 0.546$$

$$\hat{y}_1 = \frac{e^{z_1}}{\sum e^{z_j}} = \frac{1.649}{13.520} \approx 0.122$$

$$\hat{y}_2 = \frac{e^{z_2}}{\sum e^{z_j}} = \frac{4.482}{13.520} \approx 0.331$$

Prediction: The model predicts **class 0** since $\hat{y}_0 = 0.546$ is the highest probability.

Marking Rubric:

- Correct exponential calculations: 0.5 marks
 - Correct sum: 0.25 marks
 - Correct softmax probabilities for all three classes: 1.25 marks
 - Correct predicted class: 0.5 marks
- ii. CCE loss and confidence analysis

Step 1: Calculate CCE loss

$$\begin{aligned} L &= - \sum_{k=0}^2 y_k \log(\hat{y}_k) \\ &= -(0 \cdot \log(0.546) + 0 \cdot \log(0.122) + 1 \cdot \log(0.331)) \approx 1.105 \end{aligned}$$

Step 2: Analyze model confidence

- **High loss value (1.105):** Indicates poor prediction quality
- **Model is confident in wrong answer:** Assigns 54.6% probability to class 0 (wrong) vs only 33.1% to class 2 (correct)
- The loss penalizes both **wrong predictions** and **low confidence in correct class**

Marking Rubric:

- Correct CCE loss calculation: 1 mark
- Identification that model is confident in wrong prediction: 0.75 marks
- Clear explanation of confidence-loss relationship: 0.75 marks

- (b) i. Overall accuracy and precision for class 1

Overall Accuracy:

$$\begin{aligned} \text{Accuracy} &= \frac{\text{Correct predictions}}{\text{Total predictions}} \\ &= \frac{85 + 220 + 180 + 112}{800} = 0.746 = 74.6\% \end{aligned}$$

Precision for Class 1 (Negative):

From confusion matrix for class 1:

- True Positives (TP): 220 (correctly predicted as Negative when true label is Negative)
- False Positives (FP): Total predicted as class 1 minus TP, $FP = 285 - 220 = 65$

$$\text{Precision}_{\text{class 1}} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{220}{220 + 65} \approx 0.772 = 77.2\%$$

Marking Rubric:

- Correct overall accuracy calculation: 0.75 marks
 - Correct identification of TP and FP for class 1: 1 mark
 - Correct precision calculation: 0.75 marks
- ii. Confusion pattern analysis

Observations from confusion matrix:

- Class 1 → Class 2: 50 misclassifications (Negative predicted as Positive)
- Class 2 → Class 1: 45 misclassifications (Positive predicted as Negative)
- This is the highest confusion between any two classes
- These classes are semantic opposites

What this suggests about decision boundaries:

- Weak separation between Negative and Positive classes: The decision boundary between these classes is fuzzy and uncertain
- Overlapping feature distributions: Reviews with moderate sentiment language get confused
- Missing distinctive features: Model may lack features that clearly distinguish negative from positive sentiment

Is this a serious problem for business?

YES, this is quite serious:

- Opposite sentiments confused: Confusing Negative with Positive is worse than confusing adjacent classes (e.g., Negative with Very Negative)
- 95 cases total (50+45): About 12% of these two classes are confused with their opposite
- Trust implications: Stakeholders may lose confidence in the system if opposite sentiments are confused

Marking Rubric:

- Identification of weak decision boundary between opposite classes: 1 mark
- Discussion of whether this is serious for business: 1 mark
- Additional insights (causes, mitigation): 0.5 marks

(c) **Recommendation: Model B (Optimized for Critical Signs)**

i. Safety implications

- Critical signs analysis
 - Stop signs:
 - * Failure consequence: Running a stop sign causes intersection collisions, often T-bone crashes with severe injuries/fatalities
 - * Model A: 88-90% recall means missing 10-12% of stop signs
 - * Model B: 98% recall means missing only 2% of stop signs
 - * Impact: Model B reduces critical stop sign misses by 80-83%
 - No Entry signs:
 - * Failure consequence: Entering restricted areas causes wrong-way driving, head-on collisions
 - * Model A: 88-90% recall
 - * Model B: 97% recall
 - * Impact: Substantially fewer catastrophic wrong-way incidents
 - Less critical signs: **Speed Limit signs:**
 - * Model A: Higher recall (let's assume 90%)
 - * Model B: 75% recall
 - * Failure consequence: Vehicle may default to conservative speed (e.g., 25 mph) when sign is missed
 - * Impact: Slower driving, passenger inconvenience, but NOT life-threatening
 - Pedestrian Crossing:
 - * Model B: 78% recall
 - * Impact: Redundant systems provide safety backup

Risk asymmetry:

- Missing Stop/No Entry: Potential for immediate fatal accident
- Missing Speed Limit: Slower driving, no safety compromise
- The 10-15% reduction in non-critical sign detection is acceptable given the 8-10% improvement in critical sign detection

Marking Rubric:

- Clear differentiation between critical and non-critical signs: 1 mark
- Analysis of safety implications with specific consequences: 1 mark

ii. Regulatory and liability concerns

- Safety regulations – Model B's 97-98% on critical signs likely exceeds regulatory minimums, Model A's 88-90% may be borderline or below requirements
- Liability implications – Model A misses stop sign: Vehicle runs stop sign, causes collision, Model B misses speed limit sign: Vehicle drives slower
- Legal precedent considerations – Courts apply "reasonable care" standard: Did manufacturer take all reasonable precautions? Choosing Model A when Model B exists may fail this standard. Expert witnesses would testify: "Better model available, not deployed"

Overall assessment vs 2% accuracy difference:

- The 2% drop in overall accuracy (91% → 89%) is irrelevant
- What matters is performance on safety-critical classes

- Trading non-critical accuracy for critical accuracy is the right engineering trade-off
- Model B is optimized for what actually matters: preventing fatal accidents

Marking Rubric:

- Discussion of regulatory requirements for critical signs: 1 mark
- Analysis of liability implications with specific scenarios: 1 mark

(d) **Error 1 (Line 5): Wrong axis for summation**

Identification [0.75 marks]: Softmax should sum over classes (axis=1 for row-wise sum) not over batch examples (axis=0).

Correction [1 mark]:

```
return exp_Z / np.sum(exp_Z, axis=1, keepdims=True)
```

Error 2 (Line 10): Missing negative sign

Identification [0.75 marks]: The negative sign is missing.

Correction [1 mark]:

```
return -np.mean(Y_true * np.log(Y_pred))
```

Error 3 (Line 15): Using argmin instead of argmax

Identification [0.75 marks]: Prediction should select the class with the highest probability (argmax), not lowest probability (argmin).

Correction [1 mark]:

```
return np.argmax(probs, axis=1)
```

(iii) **Impact if Error 3 remains**

- Model predicts the class with the **lowest** probability. This is exactly opposite to the intended behavior
- During training: Loss and gradients are computed correctly (assuming Error 1 and 2 are fixed)
- Inverted predictions: At test time, predictions are systematically wrong
- Worst-class selection: Model consistently chooses the least likely class
- Accuracy: For K classes with roughly uniform distribution, accuracy would be approximately $1/K$ (random chance)
- Confusion matrix: Would show anti-diagonal pattern - predicting least likely class for each true class

Marking Rubric:

- Identification that predictions are inverted: 0.75 marks
- Explanation of impact on test performance: 0.75 marks

5. (a) i. ReLU activation

$$\text{ReLU}(-2.0) = \max(0, -2.0) = 0$$

$$\text{ReLU}(0.0) = \max(0, 0.0) = 0$$

$$\text{ReLU}(1.5) = \max(0, 1.5) = 1.5$$

$$\text{ReLU output} = \begin{bmatrix} 0 & 0 & 1.5 \end{bmatrix}^\top$$

Marking Rubric:

- All three ReLU outputs correct: 2 marks (0.5 + 0.5 + 1 for showing calculation)
- ii. Sigmoid activation and comparison

$$\sigma(-2.0) = \frac{1}{1 + e^{2.0}} = \frac{1}{1 + 7.389} = \frac{1}{8.389} \approx 0.119$$

$$\sigma(0.0) = \frac{1}{1 + e^0} = \frac{1}{1 + 1} = 0.5$$

$$\sigma(1.5) = \frac{1}{1 + e^{-1.5}} = \frac{1}{1 + 0.223} = \frac{1}{1.223} \approx 0.818$$

$$\text{Sigmoid output} = \begin{bmatrix} 0.119 & 0.5 & 0.818 \end{bmatrix}^\top$$

Sigmoid preserves information about negative values, while ReLU does not.

Explanation:

- ReLU behavior:
 - For $z = -2.0$: Output is 0 (information completely lost)
 - For $z = 0.0$: Output is 0
 - Cannot distinguish between "slightly negative" and "very negative"
 - All negative inputs map to the same output (0)
- Sigmoid behavior:
 - For $z = -2.0$: Output is 0.119 (preserves magnitude information)
 - For $z = 0.0$: Output is 0.5 (midpoint)
 - Different negative inputs produce different outputs in range (0, 0.5)
 - Negative information is preserved, just compressed to (0, 0.5)
- Impact on learning:
 - **ReLU**: "Dead neurons" problem - if a neuron's pre-activation is always negative, its output is always 0, and gradients become 0 (neuron stops learning)
 - **Sigmoid**: All inputs produce non-zero outputs (except $z \rightarrow -\infty$), allowing gradients to flow
 - **ReLU advantage**: Despite losing negative info, ReLU trains faster and avoids vanishing gradient problem that affects sigmoid in deep networks
 - **Trade-off**: ReLU is preferred in practice despite information loss because it's computationally efficient and enables deeper networks

Marking Rubric:

- Correct sigmoid calculations for all three values: 1.5 marks
- Correct identification that Sigmoid preserves negative information: 0.5 marks
- Clear explanation of how this affects learning: 1 mark

(b) i. **Architecture:** $[784 \rightarrow n_h \rightarrow 10]$

Parameter calculation:

$$\begin{aligned}\text{Layer 1 (Input} \rightarrow \text{Hidden):} &= 784 \times n_h + n_h \\ \text{Layer 2 (Hidden} \rightarrow \text{Output):} &= n_h \times 10 + 10 \\ \text{Parameters:} &= (784n_h + n_h) + (10n_h + 10) = 795n_h + 10 \\ \text{Constraint inequality:} &= 795n_h + 10 \leq 20000 \\ & n_h \leq 25.14\end{aligned}$$

Maximum allowed: $n_h = 25$ neurons (must be integer)

Marking Rubric:

- Correct parameter formula derivation: 1 mark
 - Correct inequality setup: 0.5 marks
 - Correct solution for maximum n_h : 1 mark
- ii. Let's choose $n_h = 20$ neurons (safely below the maximum of 25)

$$\text{Exact Parameters} = 795(20) + 10 = 15910$$

Is this reasonable given 10,000 training samples?

YES, this is reasonable:

Risk if n_h is too large:

If we chose $n_h = 25$ (maximum):

$$\text{Parameters} = 795(25) + 10 = 19885$$

$$\text{Ratio} = \frac{10000}{19885} \approx 0.50$$

- Overfitting: Model has too much capacity relative to data
- Training instability: More parameters = more opportunities for vanishing/exploding gradients
- Computational cost: Inference slower on limited hardware (mobile, embedded)
- Poor sample efficiency: Needs more data to achieve good generalization

Marking Rubric:

- Reasonable choice of n_h and correct parameter calculation: 1 mark
- Discussion of whether this is reasonable given training data: 1 mark
- Clear explanation of overfitting risk with large n_h : 0.5 marks

(c) **Recommendation: Architecture A (Lightweight)**

i. Real-time constraint and battery life

Real-time processing analysis:

Time budget:

- Requirement: 100 audio segments per second
- Time budget per segment: $\frac{1000ms}{100} = 10ms$
- Must process segment + prepare for next within 10ms

Architecture A:

- Inference time: 0.8ms per segment
- Buffer: $10ms - 0.8ms = 9.2ms$ slack time
- **Comfortably meets real-time constraint** with 92% time to spare
- Allows for:
 - OS scheduling delays
 - Background process interruptions
 - CPU throttling during high temperature
 - Future feature additions

Architecture B:

- Inference time: 3.2ms per segment
- Buffer: $10ms - 3.2ms = 6.8ms$ slack time
- **Meets constraint but with less margin**
- Risky because:
 - OS overhead can consume 1-2ms
 - Background tasks (notifications, syncs) cause delays
 - May miss segments during CPU contention
 - Dropped segments = delayed wake-word detection

Battery life impact:

Continuous operation implications:

- Voice assistant runs 24/7 waiting for wake word
- Processes $100 \text{ segments/sec} \times 3600 \text{ sec/hr} \times 24 \text{ hr} = 8.64 \text{ million inferences/day}$

Architecture A battery characteristics:

- Lower computational load (1,000 params, 0.8ms)
- CPU can stay in low power state longer
- Typical impact: 2-3% battery drain per day
- User can use phone normally

Architecture B battery characteristics:

- 4× higher computational load (4,500 params, 3.2ms)
- CPU must run at higher frequency more often
- Typical impact: 8-12% battery drain per day
- Significant user complaint potential
- May force users to disable feature

User experience implications:

- Voice assistant that drains battery gets disabled

- Defeats purpose of always-on voice detection
- User satisfaction critically depends on battery life
- 3× higher battery drain is unacceptable for 2% accuracy improvement

Marking Rubric:

- Analysis of real-time constraint with specific timing: 1 mark
- Discussion of battery life impact for continuous operation: 1 mark

ii. Accuracy improvement justification

Architecture comparison:

- Architecture A: 94% accuracy
- Architecture B: 96% accuracy
- Difference: 2 percentage points

Is 2% accuracy improvement worth it?

NO - The improvement does NOT justify the costs:

Practical impact of 2% accuracy difference:

- False negative rate:
 - Architecture A: 6% of speech segments misclassified as silence
 - Architecture B: 4% of speech segments misclassified as silence
 - Difference: 2 percentage points
- Real-world impact:
 - Wake word typically requires 0.5-1 second of speech (50-100 segments)
 - Actual wake word detection rate: A $\approx$ 98%, B $\approx$ 99% (not 94% vs 96%)
 - User-perceived difference is minimal

Cost analysis:

- Architecture B costs:
 - 4× slower inference (3.2ms vs 0.8ms)
 - 4.5× more parameters (4,500 vs 1,000)
 - 4.5× more memory (18KB vs 4KB)
 - 3-4× higher battery drain
- Architecture B benefits:
 - 2% improvement in segment-level accuracy
 - Marginal improvement in wake word detection
 - Negligible user experience improvement
- **Value proposition:** Costs far outweigh benefits

Marking Rubric:

- Clear statement of whether 2% justifies costs: 0.5 marks
- Analysis of practical impact of accuracy difference: 1 mark
- Discussion of trade-offs (battery vs accuracy): 0.5 marks

(d) **Error 1 (Line 8): ReLU derivative at $z=0$**

Identification [0.75 marks]:

Correction [1 mark]:

```
return (Z > 0).astype(float)
```

Explanation: While the difference at exactly $z = 0$ seems minor, using $\geq$ can cause subtle issues where zero activations still receive gradients, leading to unnecessary computation and potential numerical instability.

Error 2 (Line 14): Wrong sign in output gradient

Identification [0.75 marks]:

Correction [1 mark]:

$$dZ2 = A2 - Y$$

Error 3 (Line 24): Weight update sign reversed for W1

Identification [0.75 marks]:

Correction [1 mark]:

$$W1 = W1 - lr * dW1$$

(iii) Impact if Error 2 remains

- The sign error propagates through all subsequent gradient calculations
 - When $\hat{y} > y$ (over-prediction):
 - * Correct: $dW2 > 0$, so $W2$ decreases, reducing $\hat{y}$
 - * Incorrect: $dW2$ has wrong sign contribution from $+y$, gradient distorted
 - When $\hat{y} < y$ (under-prediction):
 - * Correct: $dW2 < 0$, so $W2$ increases, increasing $\hat{y}$
 - * Incorrect: $dW2$ has wrong sign, magnitude wrong
- Loss increases instead of decreasing: Weights move to increase error, not reduce it
- Training diverges: Loss explodes to infinity
- No learning occurs: Model performance remains at random chance
- Oscillations possible: If learning rate is small, loss may oscillate wildly without converging

Marking Rubric:

- Identification that gradient has wrong sign/direction: 0.75 marks
- Explanation of how this corrupts backpropagation through layers: 0.75 marks